Лабораторная работа №7

Коллективные операции в MPI

Среда выполнения: Ubuntu в WSL2, Open MPI 5.0.10, один компьютер, 8 логических процессоров. Язык C; локальные и итоговые суммы - long long (MPI_LONG_LONG_INT).

```
mpicc -O2 -Wall -Wextra mpi_array_sum.c -o mpi_array_sum
mpiexec --oversubscribe -n 4 ./mpi_array_sum 1000000
mpiexec --oversubscribe -n 4 ./mpi_array_sum 1000000 10000
```

1. Алгоритм и назначение операций

Вычисляется сумма массива a[i] = i + 1, i = 0, ..., N - 1. Ожидаемый результат S = N(N + 1)/2. Ранг 0 читает N; MPI_Bcast передаёт параметры и признак их корректности всем процессам. При N > 0 и N mod P = 0 процесс 0 создаёт массив. MPI_Scatter выдаёт каждому рангу r непрерывный блок из N/P элементов с индекса rN/P; каждый процесс вычисляет локальную сумму.

MPI_Gather собирает P локальных сумм на ранге 0, где они складываются циклом. MPI_Reduce с MPI_SUM возвращает общую сумму рангу 0. MPI_Allreduce возвращает её всем процессам, и каждый проверяет свой результат. Все ранги выполняют коллективные вызовы в одинаковом порядке и завершают работу через MPI_Finalize. Неверные параметры отклоняются согласованно.

2. Проверка корректности

Число процессов P	Размер массива N	Ожидаемая сумма	Результат Gather	Результат Reduce	Результат Allreduce	Корректно?
2	100	5050	5050	5050	5050	Да
4	100	5050	5050	5050	5050	Да
4	10000	50005000	50005000	50005000	50005000	Да
8	1000000	500000500000	500000500000	500000500000	500000500000	Да

Во всех конфигурациях результаты трёх операций совпали с формулой. Проверки N = P и неверных аргументов также пройдены; при ошибках программа завершается без зависания остальных процессов.

3. Влияние размера массива

Для каждой конфигурации усреднены пять отдельных запусков. Измерение MPI_Wtime начинается перед MPI_Bcast и заканчивается после основного MPI_Allreduce. Оно включает выделение памяти, заполнение массива, распределение данных, локальную сумму и все три способа агрегирования. Время - максимум по рангам; запуск MPI, проверка и вывод исключены.

Число процессов P	Размер массива N	Время выполнения, с	Результат корректен?
4	10000	0.000186250	Да
4	100000	0.000522146	Да
4	1000000	0.004251140	Да
4	10000000	0.070732933	Да

Среднее время увеличивается с размером массива: растут затраты на заполнение, передачу и обработку элементов.

4. Влияние числа процессов

Размер массива N = 1000000. Ускорение $S_P = T_1/T_P$, эффективность $E_P = S_P/P$. База T_1 - среднее время пяти запусков этой же программы с одним MPI-процессом.

Размер массива N	Число процессов P	Время выполнения, с	Ускорение S_P	Эффективность E_P
1000000	1	0.009815906	1.00	1.00
1000000	2	0.011160223	0.88	0.44
1000000	4	0.004472177	2.19	0.55
1000000	8	0.014961967	0.66	0.08

В этой серии минимальное среднее время получено при P = 4 (S = 2.19, E = 0.55). Последовательное заполнение массива на ранге 0, распределение данных и коллективные операции ограничивают ускорение. При восьми процессах среднее время выше, чем при четырёх; увеличение числа процессов не гарантирует ускорения.

5. Сравнение Gather и Reduce

Для P = 2, 4, 8 выполнено пять серий по 10000 повторений после 20 прогревочных вызовов. В каждой серии берётся максимум среднего времени операции по рангам, затем результаты серий усредняются. Для Gather учтено сложение на ранге 0; в обоих вариантах включена проверка каждой суммы.

Число процессов P	Способ агрегирования	Число повторений	Среднее время операции, с	Результат корректен?
2	Gather + сумма на процессе 0	10000	0.000000311	Да
2	Reduce	10000	0.000000251	Да
4	Gather + сумма на процессе 0	10000	0.000000835	Да
4	Reduce	10000	0.000000557	Да
8	Gather + сумма на процессе 0	10000	0.000003299	Да
8	Reduce	10000	0.000002446	Да

Оба способа дали правильный результат. В этой серии Reduce быстрее Gather с последующим сложением для всех трёх значений P. Времена малы и зависят от среды запуска, поэтому результат не определяет универсально самую быструю операцию.

Вывод

Gather нужен, когда корню требуются отдельные результаты процессов; для получения суммы нужны дополнительный буфер из P элементов и цикл. Reduce выражает объединение одним вызовом и возвращает результат только корню. Allreduce удобен, когда общий результат нужен каждому процессу для дальнейших вычислений. Проверки подтвердили корректность всех трёх операций. Замеры описывают локальные запуски в WSL и не характеризуют производительность кластера.